

Team No 108 - Tenant Rent Collection System

Name: Shraddha Chandrashekhar Hade (TL) - 250840320188

Shruti Dhanalal Wadile (AL) - 250840320191

1. ER Diagram Overview

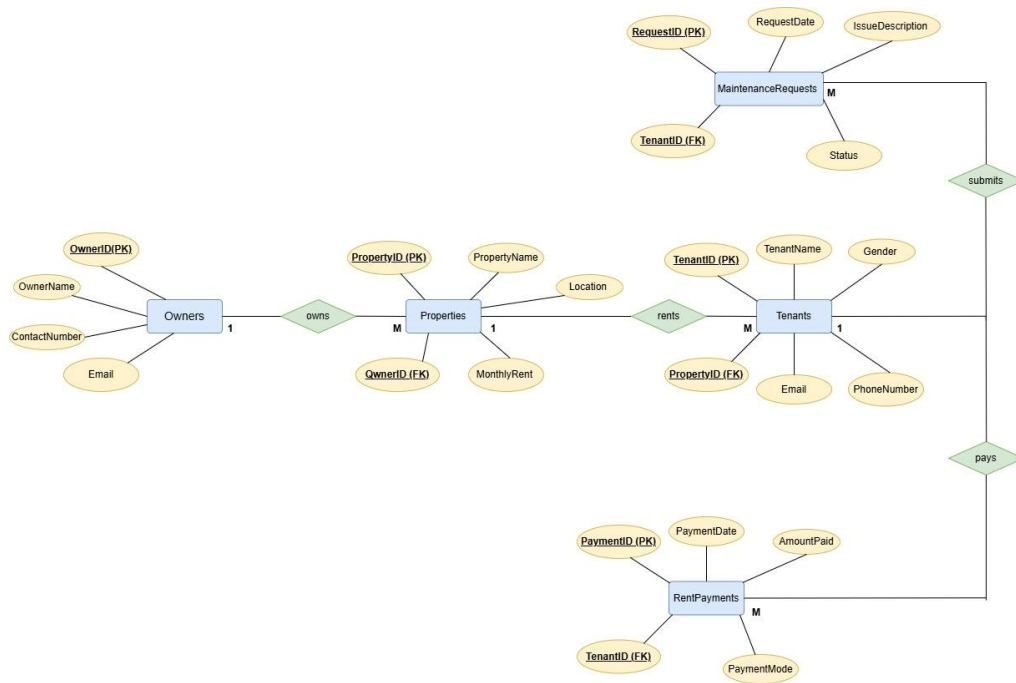
Entities:

- Tenant
- Property
- RentPayment
- Owner
- MaintenanceRequest

Relationships:

- A Property belongs to one Owner (M-1)
- A Tenant rents one Property (M-1)
- A Tenant makes many RentPayments (1-M)
- A Property can have many RentPayments through its tenants (1-M)

ER Diagram:



2. Database Schema (DDL)

```
create table Owners(  
  OwnerID int primary key auto_increment,  
  OwnerName varchar(100) not null,  
  ContactNumber varchar(15),  
  Email varchar(100) unique  
);
```

```
mysql> desc owners;
```

Field	Type	Null	Key	Default	Extra
OwnerID	int	NO	PRI	NULL	auto_increment
OwnerName	varchar(100)	NO		NULL	
ContactNumber	varchar(15)	YES		NULL	
Email	varchar(100)	YES	UNI	NULL	

```
4 rows in set (0.00 sec)
```

```
create table Properties(  
  PropertyID int primary key auto_increment,  
  PropertyName varchar(100) not null,  
  Location varchar(100) not null,  
  MonthlyRent decimal(10,2) not null check (MonthlyRent > 0),  
  OwnerID int,  
  foreign key (OwnerID) references Owners(OwnerID)  
);
```

```
mysql> desc Properties;
```

Field	Type	Null	Key	Default	Extra
PropertyID	int	NO	PRI	NULL	auto_increment
PropertyName	varchar(100)	NO		NULL	
Location	varchar(100)	NO		NULL	
MonthlyRent	decimal(10,2)	NO		NULL	
OwnerID	int	YES	MUL	NULL	

```
5 rows in set (0.00 sec)
```

```

create table Tenants(
  TenantID int primary key auto_increment,
  TenantName varchar(100) not null,
  Gender enum('Male','Female','Other'),
  PhoneNumber varchar(15) unique,
  Email varchar(100),
  PropertyID int,
  foreign key (PropertyID) references Properties(PropertyID)
);

```

```

mysql> desc Tenants;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| TenantID | int | NO | PRI | NULL | auto_increment |
| TenantName | varchar(100) | NO | | NULL | |
| Gender | enum('Male','Female','Other') | YES | | NULL | |
| PhoneNumber | varchar(15) | YES | UNI | NULL | |
| Email | varchar(100) | YES | | NULL | |
| PropertyID | int | YES | MUL | NULL | |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.00 sec)

```

```

create table RentPayments(
  PaymentID int primary key auto_increment,
  TenantID int,
  PaymentDate date not null,
  AmountPaid decimal(10,2) not null check (AmountPaid >= 0),
  PaymentMode enum('Cash','Online','Cheque'),
  foreign key (TenantID) references Tenants(TenantID)
);

```

```

mysql> desc RentPayments;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| PaymentID | int | NO | PRI | NULL | auto_increment |
| TenantID | int | YES | MUL | NULL | |
| PaymentDate | date | NO | | NULL | |
| AmountPaid | decimal(10,2) | NO | | NULL | |
| PaymentMode | enum('Cash','OnLine','Cheque') | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

```

```
create table MaintenanceRequests (
  RequestID int primary key auto_increment,
  TenantID int,
  RequestDate date default (curdate()),
  IssueDescription text,
  Status enum('Pending', 'In Progress', 'Resolved') default 'Pending',
  foreign key (TenantID) references Tenants(TenantID)
);
```

```
mysql> desc MaintenanceRequests;
```

Field	Type	Null	Key	Default	Extra
RequestID	int	NO	PRI	NULL	auto_increment
TenantID	int	YES	MUL	NULL	
RequestDate	date	YES		curdate()	DEFAULT_GENERATED
IssueDescription	text	YES		NULL	
Status	enum('Pending', 'In Progress', 'Resolved')	YES		Pending	

5 rows in set (0.00 sec)

3. DML Operations (INSERT, UPDATE, DELETE)

Owners Table:

insert into Owners values

```
(1, 'Ramesh Patel', '9812651239', 'ramesh@gmail.com'),
(2, 'Meenakshi Patil', '973919834', 'meenakshi@gmail.com'),
(3, 'Jay Aggrawal', '9129816542', 'jay@gmail.com'),
(4, 'Ashish Bhatiya', '9639813473', 'ashish@gmail.com'),
(5, 'Rahul Singh', '9167196527', 'rahul@gmail.com'),
(6, 'Ketki Joshi', '9278915239', 'ketki@gmail.com'),
(7, 'Sanvi Mathur', '9618239046', 'sanvi@gmail.com'),
(8, 'Laxmi Bajaj', '9928461285', 'laxmi@gmail.com'),
(9, 'Arvind Rao', '9812934782', 'arvind@gmail.com'),
(10, 'Deepika Singh', '9239845713', 'deepika@gmail.com');
```

```
mysql> select * from Owners;
```

OwnerID	OwnerName	ContactNumber	Email
1	Ramesh Patel	9812651239	ramesh@gmail.com
2	Meenakshi Patil	973919834	meenakshi@gmail.com
3	Jay Aggrawal	9129816542	jay@gmail.com
4	Ashish Bhatiya	9639813473	ashish@gmail.com
5	Rahul Singh	9876543210	rahul@gmail.com
6	Ketki Joshi	9278915239	ketki@gmail.com
7	Sanvi Mathur	9618239046	sanvi@gmail.com
8	Laxmi Bajaj	9928461285	laxmi@gmail.com
9	Arvind Rao	9812934782	arvind@gmail.com
10	Deepika Singh	9239845713	deepika@gmail.com

10 rows in set (0.00 sec)

Properties Table:

insert into Properties(PropertyID, PropertyName, Location, MonthlyRent, OwnerID) values

(101, 'Sunrise Apartments', 'Mumbai', 12000.00, 1),
(102, 'Green Valley Homes', 'Pune', 15000.00, 2),
(103, 'Ocean View Residency', 'Goa', 18000.00, 3),
(104, 'Hilltop Towers', 'Lonavala', 14000.00, 4),
(105, 'Maple Residency', 'Delhi', 20000.00, 5),
(106, 'Royal Gardens', 'Bangalore', 25000.00, 6),
(107, 'Riverstone Villas', 'Hyderabad', 17000.00, 7),
(108, 'Golden Nest Apartments', 'Chennai', 15500.00, 8),
(109, 'Palm Paradise', 'Jaipur', 16000.00, 9),
(110, 'Silverline Heights', 'Ahmedabad', 22000.00, 10);

```
mysql> select * from Properties;
```

PropertyID	PropertyName	Location	MonthlyRent	OwnerID
101	Sunrise Apartments	Mumbai	12000.00	1
102	Green Valley Homes	Pune	15000.00	2
103	Ocean View Residency	Goa	18000.00	3
104	Hilltop Towers	Lonavala	14000.00	4
105	Maple Residency	Delhi	20000.00	5
106	Royal Gardens	Bangalore	25000.00	6
107	Riverstone Villas	Hyderabad	17000.00	7
108	Golden Nest Apartments	Chennai	15500.00	8
109	Palm Paradise	Jaipur	16000.00	9
110	Silverline Heights	Ahmedabad	22000.00	10

10 rows in set (0.09 sec)

Tenants Table:

insert into Tenants values

(1, 'Amit Verma', 'Male', '9811112233', 'amitv@gmail.com', 101),
(2, 'Riya Shah', 'Female', '9822334455', 'riyashah@gmail.com', 102),
(3, 'Karan Patel', 'Male', '9833445566', 'karanp@gmail.com', 103),
(4, 'Sneha Mehta', 'Female', '9844556677', 'snehamehta@gmail.com', 104),
(5, 'Arjun Das', 'Male', '9855667788', 'arjundas@gmail.com', 105),
(6, 'Priya Nair', 'Female', '9866778899', 'priyanair@gmail.com', 106),
(7, 'Rohit Gupta', 'Male', '9877889900', 'rohitgupta@gmail.com', 107),
(8, 'Nisha Rao', 'Female', '9888990011', 'nisharao@gmail.com', 108),
(9, 'Vikas Sharma', 'Male', '9899001122', 'vikassharma@gmail.com', 109),
(10, 'Aarohi Joshi', 'Female', '9900112233', 'aarahijoshi@gmail.com', 110);

```
mysql> select * from Tenants;
```

TenantID	TenantName	Gender	PhoneNumber	Email	PropertyID
1	Amit Verma	Male	9811112233	amitv@gmail.com	101
2	Riya Shah	Female	9822334455	riyashah@gmail.com	102
3	Karan Patel	Male	9833445566	karanp@gmail.com	103
4	Sneha Mehta	Female	9844556677	snehamehta@gmail.com	104
5	Arjun Das	Male	9855667788	arjundas@gmail.com	105
6	Priya Nair	Female	9866778899	priyanair@gmail.com	106
7	Rohit Gupta	Male	9877889900	rohitgupta@gmail.com	107
8	Nisha Rao	Female	9888990011	nisharao@gmail.com	108
9	Vikas Sharma	Male	9899001122	vikassharma@gmail.com	109
10	Aarohi Joshi	Female	9900112233	aarohijoshi@gmail.com	110

```
10 rows in set (0.01 sec)
```

RentPayments Table:

PaymentID will auto-increment automatically:-

insert into RentPayments (TenantID, PaymentDate, AmountPaid, PaymentMode) values

```
(1, '2025-10-01', 12000.00, 'Online'),
(2, '2025-10-03', 15000.00, 'Cash'),
(3, '2025-09-28', 17500.00, 'Cheque'),
(4, '2025-10-02', 14200.00, 'Online'),
(5, '2025-10-04', 20500.00, 'Cash'),
(6, '2025-09-30', 24750.00, 'Online'),
(7, '2025-10-05', 16800.00, 'Cash'),
(8, '2025-10-06', 15800.00, 'Cheque'),
(9, '2025-10-07', 16250.00, 'Online'),
(10, '2025-10-08', 22500.00, 'Cash');
```

```
mysql> select * from RentPayments;
```

PaymentID	TenantID	PaymentDate	AmountPaid	PaymentMode
1	1	2025-10-01	12000.00	Online
2	2	2025-10-03	15000.00	Online
3	3	2025-09-28	17500.00	Cheque
4	4	2025-10-02	14200.00	Online
5	5	2025-10-04	20500.00	Cash
6	6	2025-09-30	24750.00	Online
7	7	2025-10-05	16800.00	Cash
8	8	2025-10-06	15800.00	Cheque
9	9	2025-10-07	16250.00	Online
10	10	2025-10-08	22500.00	Cash

```
10 rows in set (0.01 sec)
```

MaintenanceRequests Table:

insert into MaintenanceRequests (TenantID, RequestDate, IssueDescription, Status)
values

```
(1, '2025-09-25', 'Leaking tap in kitchen', 'Resolved'),  
(2, '2025-09-28', 'Air conditioner not cooling', 'In Progress'),  
(3, '2025-10-01', 'Broken window in living room', 'Pending'),  
(4, '2025-10-03', 'Power outage in bedroom', 'Resolved'),  
(5, '2025-10-04', 'Water leakage in bathroom', 'Pending'),  
(6, '2025-10-05', 'Door lock issue', 'Resolved'),  
(7, '2025-10-06', 'Internet connection unstable', 'In Progress'),  
(8, '2025-10-07', 'Wall paint peeling off', 'Pending'),  
(9, '2025-10-08', 'Ceiling fan making noise', 'Resolved'),  
(10, '2025-10-09', 'Kitchen sink clogged', 'In Progress');
```

```
mysql> select * from MaintenanceRequests;  
+-----+-----+-----+-----+-----+  
| RequestID | TenantID | RequestDate | IssueDescription | Status |  
+-----+-----+-----+-----+-----+  
| 1 | 1 | 2025-09-25 | Leaking tap in kitchen | Resolved |  
| 2 | 2 | 2025-09-28 | Air conditioner not cooling | In Progress |  
| 3 | 3 | 2025-10-01 | Broken window in living room | Pending |  
| 4 | 4 | 2025-10-03 | Power outage in bedroom | Resolved |  
| 5 | 5 | 2025-10-04 | Water leakage in bathroom | Pending |  
| 6 | 6 | 2025-10-05 | Door lock issue | Resolved |  
| 7 | 7 | 2025-10-06 | Internet connection unstable | In Progress |  
| 8 | 8 | 2025-10-07 | Wall paint peeling off | Pending |  
| 9 | 9 | 2025-10-08 | Ceiling fan making noise | Resolved |  
| 10 | 10 | 2025-10-09 | Kitchen sink clogged | In Progress |  
+-----+-----+-----+-----+-----+  
10 rows in set (0.00 sec)
```

UPDATE Queries:

1. Increase Monthly Rent

Problem: Increase the monthly rent by ₹2000 for all properties located in *Mumbai*.

update Properties set MonthlyRent = MonthlyRent + 2000 where Location = 'Mumbai';

```
mysql> select * from Properties;  
+-----+-----+-----+-----+-----+  
| PropertyID | PropertyName | Location | MonthlyRent | OwnerID |  
+-----+-----+-----+-----+-----+  
| 101 | Sunrise Apartments | Mumbai | 12000.00 | 1 |  
| 102 | Green Valley Homes | Pune | 15000.00 | 2 |  
| 103 | Ocean View Residency | Goa | 18000.00 | 3 |  
| 104 | Hilltop Towers | Lonavala | 14000.00 | 4 |  
| 105 | Maple Residency | Delhi | 20000.00 | 5 |  
| 106 | Royal Gardens | Bangalore | 25000.00 | 6 |  
| 107 | Riverstone Villas | Hyderabad | 17000.00 | 7 |  
| 108 | Golden Nest Apartments | Chennai | 15500.00 | 8 |  
| 109 | Palm Paradise | Jaipur | 16000.00 | 9 |  
| 110 | Silverline Heights | Ahmedabad | 22000.00 | 10 |  
+-----+-----+-----+-----+-----+  
10 rows in set (0.09 sec)
```

```
mysql> select * from Properties;
```

PropertyID	PropertyName	Location	MonthlyRent	OwnerID
101	Sunrise Apartments	Mumbai	14000.00	1
102	Green Valley Homes	Pune	15000.00	2
103	Ocean View Residency	Goa	18000.00	3
104	Hilltop Towers	Lonavala	14000.00	4
105	Maple Residency	Delhi	20000.00	5
106	Royal Gardens	Bangalore	25000.00	6
107	Riverstone Villas	Hyderabad	17000.00	7
108	Golden Nest Apartments	Chennai	15500.00	8
109	Palm Paradise	Jaipur	16000.00	9
110	Silverline Heights	Ahmedabad	22000.00	10

```
10 rows in set (0.00 sec)
```

2. Change Owner's Contact Number

Problem: Update the phone number of the owner named *Rahul Singh*.

update Owners set ContactNumber = '9876543210' where OwnerName = 'Rahul Singh';

```
mysql> select * from Owners;
```

OwnerID	OwnerName	ContactNumber	Email
1	Ramesh Patel	9812651239	ramesh@gmail.com
2	Meenakshi Patil	973919834	meenakshi@gmail.com
3	Jay Aggrawal	9129816542	jay@gmail.com
4	Ashish Bhatiya	9639813473	ashish@gmail.com
5	Rahul Singh	9876543210	rahul@gmail.com
6	Ketki Joshi	9278915239	ketki@gmail.com
7	Sanvi Mathur	9618239046	sanvi@gmail.com
8	Laxmi Bajaj	9928461285	laxmi@gmail.com
9	Arvind Rao	9812934782	arvind@gmail.com
10	Deepika Singh	9239845713	deepika@gmail.com

```
10 rows in set (0.00 sec)
```

3. Update Payment Mode

Problem: Change payment mode to *Online* for all tenants who paid more than ₹13000.

update RentPayments set PaymentMode = 'Online' WHERE AmountPaid > 13000;

```
mysql> select * from RentPayments;
```

PaymentID	TenantID	PaymentDate	AmountPaid	PaymentMode
1	1	2025-10-01	12000.00	Online
2	2	2025-10-03	15000.00	Online
3	3	2025-09-28	17500.00	Cheque
4	4	2025-10-02	14200.00	Online
5	5	2025-10-04	20500.00	Cash
6	6	2025-09-30	24750.00	Online
7	7	2025-10-05	16800.00	Cash
8	8	2025-10-06	15800.00	Cheque
9	9	2025-10-07	16250.00	Online
10	10	2025-10-08	22500.00	Cash

```
10 rows in set (0.01 sec)
```

```
mysql> select * from RentPayments;
```

PaymentID	TenantID	PaymentDate	AmountPaid	PaymentMode
1	1	2025-10-01	12000.00	Online
2	2	2025-10-03	15000.00	Online
3	3	2025-09-28	17500.00	Online
4	4	2025-10-02	14200.00	Online
5	5	2025-10-04	20500.00	Online
6	6	2025-09-30	24750.00	Online
7	7	2025-10-05	16800.00	Online
8	8	2025-10-06	15800.00	Online
9	9	2025-10-07	16250.00	Online
10	10	2025-10-08	22500.00	Online

```
10 rows in set (0.00 sec)
```

4. Modify Tenant's Property

Problem: Reassign *Riya Shah* to Property ID 5.

update Tenants set PropertyID = 105 where TenantName = 'Riya Shah';

```
mysql> select * from Tenants;
```

TenantID	TenantName	Gender	PhoneNumber	Email	PropertyID
1	Amit Verma	Male	9811112233	amitv@gmail.com	101
2	Riya Shah	Female	9822334455	riyashah@gmail.com	102
3	Karan Patel	Male	9833445566	karanp@gmail.com	103
4	Sneha Mehta	Female	9844556677	snehamehta@gmail.com	104
5	Arjun Das	Male	9855667788	arjundas@gmail.com	105
6	Priya Nair	Female	9866778899	priyanair@gmail.com	106
7	Rohit Gupta	Male	9877889900	rohitgupta@gmail.com	107
8	Nisha Rao	Female	9888990011	nisharao@gmail.com	108
9	Vikas Sharma	Male	9899001122	vikassharma@gmail.com	109
10	Aarohi Joshi	Female	9900112233	aarohijoshi@gmail.com	110

```
10 rows in set (0.00 sec)
```

```
mysql> select * from Tenants;
```

TenantID	TenantName	Gender	PhoneNumber	Email	PropertyID
1	Amit Verma	Male	9811112233	amitv@gmail.com	101
2	Riya Shah	Female	9822334455	riyashah@gmail.com	105
3	Karan Patel	Male	9833445566	karanp@gmail.com	103
4	Sneha Mehta	Female	9844556677	snehamehta@gmail.com	104
5	Arjun Das	Male	9855667788	arjundas@gmail.com	105
6	Priya Nair	Female	9866778899	priyanair@gmail.com	106
7	Rohit Gupta	Male	9877889900	rohitgupta@gmail.com	107
8	Nisha Rao	Female	9888990011	nisharao@gmail.com	108
9	Vikas Sharma	Male	9899001122	vikassharma@gmail.com	109
10	Aarohi Joshi	Female	9900112233	aarohijoshi@gmail.com	110

```
10 rows in set (0.00 sec)
```

5. Update Maintenance Request Status

Problem: Mark all *Pending* maintenance requests as *In Progress* for Tenant ID 8.

UPDATE MaintenanceRequests set Status = 'In Progress' where TenantID = 8 and Status = 'Pending';

```
mysql> select * from MaintenanceRequests;
```

RequestID	TenantID	RequestDate	IssueDescription	Status
1	1	2025-09-25	Leaking tap in kitchen	Resolved
2	2	2025-09-28	Air conditioner not cooling	In Progress
3	3	2025-10-01	Broken window in living room	Pending
4	4	2025-10-03	Power outage in bedroom	Resolved
5	5	2025-10-04	Water leakage in bathroom	Pending
6	6	2025-10-05	Door lock issue	Resolved
7	7	2025-10-06	Internet connection unstable	In Progress
8	8	2025-10-07	Wall paint peeling off	Pending
9	9	2025-10-08	Ceiling fan making noise	Resolved
10	10	2025-10-09	Kitchen sink clogged	In Progress

10 rows in set (0.00 sec)

```
mysql> select * from MaintenanceRequests;
```

RequestID	TenantID	RequestDate	IssueDescription	Status
1	1	2025-09-25	Leaking tap in kitchen	Resolved
2	2	2025-09-28	Air conditioner not cooling	In Progress
3	3	2025-10-01	Broken window in living room	Pending
4	4	2025-10-03	Power outage in bedroom	Resolved
5	5	2025-10-04	Water leakage in bathroom	Pending
6	6	2025-10-05	Door lock issue	Resolved
7	7	2025-10-06	Internet connection unstable	In Progress
8	8	2025-10-07	Wall paint peeling off	In Progress
9	9	2025-10-08	Ceiling fan making noise	Resolved
10	10	2025-10-09	Kitchen sink clogged	In Progress

10 rows in set (0.00 sec)

DELETE Queries:

1. Remove a Tenant Record

Problem: Delete tenant *Amit Verma* from the Tenants table.

delete from Tenants where TenantName = 'Amit Verma';

2. Delete Old Maintenance Requests

Problem: Remove all maintenance requests made before 2025-09-30.(renamed the table before)

delete from ServiceRequests where RequestDate < '2025-09-30';

```
mysql> select * from ServiceRequests;
```

RequestID	TenantID	RequestDate	IssueDescription	Status
1	1	2025-09-25	Leaking tap in kitchen	Resolved
2	2	2025-09-28	Air conditioner not cooling	In Progress
3	3	2025-10-01	Broken window in living room	Pending
4	4	2025-10-03	Power outage in bedroom	Resolved
5	5	2025-10-04	Water leakage in bathroom	Pending
6	6	2025-10-05	Door lock issue	Resolved
7	7	2025-10-06	Internet connection unstable	In Progress
8	8	2025-10-07	Wall paint peeling off	Pending
9	9	2025-10-08	Ceiling fan making noise	Resolved
10	10	2025-10-09	Kitchen sink clogged	In Progress

10 rows in set (0.00 sec)

```
mysql> select * from ServiceRequests;
```

RequestID	TenantID	RequestDate	IssueDescription	Status
3	3	2025-10-01	Broken window in living room	Pending
4	4	2025-10-03	Power outage in bedroom	Resolved
5	5	2025-10-04	Water leakage in bathroom	Pending
6	6	2025-10-05	Door lock issue	Resolved
7	7	2025-10-06	Internet connection unstable	In Progress
8	8	2025-10-07	Wall paint peeling off	Pending
9	9	2025-10-08	Ceiling fan making noise	Resolved
10	10	2025-10-09	Kitchen sink clogged	In Progress

8 rows in set (0.00 sec)

3. Remove a Property Record

Problem: Delete the property *Green Valley Homes* from the Properties table.

delete from Properties where PropertyName = 'Green Valley Homes';

```
mysql> select * from Properties;
```

PropertyID	PropertyName	Location	MonthlyRent	OwnerID
101	Sunrise Apartments	Mumbai	12000.00	1
103	Ocean View Residency	Goa	18000.00	3
104	Hilltop Towers	Lonavala	14000.00	4
105	Maple Residency	Delhi	20000.00	5
106	Royal Gardens	Bangalore	25000.00	6
107	Riverstone Villas	Hyderabad	17000.00	7
108	Golden Nest Apartments	Chennai	15500.00	8
109	Palm Paradise	Jaipur	16000.00	9
110	Silverline Heights	Ahmedabad	22000.00	10

9 rows in set (0.00 sec)

ALTER Queries:

1. Add a New Column

Problem: Add a new column Occupation to the Tenants table to store each tenant's profession.

ALTER TABLE Tenants ADD Occupation VARCHAR(50);

```
mysql> ALTER TABLE Tenants ADD Occupation VARCHAR(50);
Query OK, 0 rows affected (0.14 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> desc Tenants;
```

Field	Type	Null	Key	Default	Extra
TenantID	int	NO	PRI	NULL	auto_increment
TenantName	varchar(100)	NO		NULL	
Gender	enum('Male', 'Female', 'Other')	YES		NULL	
PhoneNumber	varchar(15)	YES	UNI	NULL	
Email	varchar(100)	YES		NULL	
PropertyID	int	YES	MUL	NULL	
Occupation	varchar(50)	YES		NULL	

```
7 rows in set (0.05 sec)
```

2. Drop a Column

Problem: Remove the Occupation column from the Tenants table (if no longer needed).

ALTER TABLE Tenants DROP COLUMN Occupation;

```
mysql> ALTER TABLE Tenants DROP COLUMN Occupation;
Query OK, 0 rows affected (0.08 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> desc Tenants;
```

Field	Type	Null	Key	Default	Extra
TenantID	int	NO	PRI	NULL	auto_increment
TenantName	varchar(100)	NO		NULL	
Gender	enum('Male', 'Female', 'Other')	YES		NULL	
PhoneNumber	varchar(15)	YES	UNI	NULL	
Email	varchar(100)	YES		NULL	
PropertyID	int	YES	MUL	NULL	

```
6 rows in set (0.01 sec)
```

3. Rename a Table

Problem: Rename the table MaintenanceRequests to ServiceRequests.

alter table MaintenanceRequests rename to ServiceRequests;

```
mysql> alter table MaintenanceRequests rename to ServiceRequests;
Query OK, 0 rows affected (0.07 sec)
```

```
mysql> desc ServiceRequests;
```

Field	Type	Null	Key	Default	Extra
RequestID	int	NO	PRI	NULL	auto_increment
TenantID	int	YES	MUL	NULL	
RequestDate	date	YES		curdate()	DEFAULT_GENERATED
IssueDescription	text	YES		NULL	
Status	enum('Pending','In Progress','Resolved')	YES		Pending	

5 rows in set (0.00 sec)

4. Add an Index

Problem: Improve search performance for the Location column in the Properties table.

alter table Properties add index idx_location (Location);

```
mysql> show indexes from Properties;
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
properties	0	PRIMARY	1	PropertyID	A	10	NULL	NULL	YES	BTREE			YES	NULL
properties	1	OwnerID	1	OwnerID	A	10	NULL	NULL	YES	BTREE			YES	NULL
properties	1	idx_location	1	Location	A	10	NULL	NULL		BTREE			YES	NULL

3 rows in set (0.05 sec)

4. SQL Queries

WHERE CLAUSE:

1. Find Male Tenants

SELECT TenantName, Gender FROM Tenants WHERE Gender = 'Male';

```
mysql> SELECT TenantName, Gender FROM Tenants WHERE Gender = 'Male';
```

TenantName	Gender
Amit Verma	Male
Karan Patel	Male
Arjun Das	Male
Rohit Gupta	Male
Vikas Sharma	Male

5 rows in set (0.00 sec)

2. Find Tenants Whose Name Starts with 'S'

SELECT TenantID, TenantName FROM Tenants WHERE TenantName LIKE 'S%';

```
mysql> SELECT TenantID, TenantName FROM Tenants WHERE TenantName LIKE 'S%';
+-----+-----+
| TenantID | TenantName |
+-----+-----+
|         4 | Sneha Mehta |
+-----+-----+
1 row in set (0.00 sec)
```

3. Find Tenants with a Specific Property Assigned

SELECT TenantName, PropertyID FROM Tenants WHERE PropertyID = 103;

```
mysql> SELECT TenantName, PropertyID FROM Tenants WHERE PropertyID = 103;
+-----+-----+
| TenantName | PropertyID |
+-----+-----+
| Karan Patel |         103 |
+-----+-----+
1 row in set (0.00 sec)
```

4. Find Tenants Whose Email Ends with '@gmail.com'

SELECT TenantName, Email FROM Tenants WHERE Email LIKE '%@gmail.com';

```
mysql> SELECT TenantName, Email FROM Tenants WHERE Email LIKE '%@gmail.com';
+-----+-----+
| TenantName | Email |
+-----+-----+
| Amit Verma | amitv@gmail.com |
| Riya Shah | riyashah@gmail.com |
| Karan Patel | karanp@gmail.com |
| Sneha Mehta | snehamehta@gmail.com |
| Arjun Das | arjundas@gmail.com |
| Priya Nair | priyanair@gmail.com |
| Rohit Gupta | rohitgupta@gmail.com |
| Nisha Rao | nisharao@gmail.com |
| Vikas Sharma | vikassharma@gmail.com |
| Aarohi Joshi | aarohijoshi@gmail.com |
+-----+-----+
10 rows in set (0.00 sec)
```

5. Find Tenants Who Have Provided a Phone Number

SELECT TenantName, PhoneNumber FROM Tenants WHERE PhoneNumber IS NOT NULL;

```
mysql> SELECT TenantName, PhoneNumber FROM Tenants WHERE PhoneNumber IS NOT NULL;
+-----+-----+
| TenantName | PhoneNumber |
+-----+-----+
| Amit Verma | 9811112233 |
| Riya Shah | 9822334455 |
| Karan Patel | 9833445566 |
| Sneha Mehta | 9844556677 |
| Arjun Das | 9855667788 |
| Priya Nair | 9866778899 |
| Rohit Gupta | 9877889900 |
| Nisha Rao | 9888990011 |
| Vikas Sharma | 9899001122 |
| Aarohi Joshi | 9900112233 |
+-----+-----+
10 rows in set (0.00 sec)
```

GROUP BY:

1. Group Tenants by Gender

SELECT Gender, COUNT(TenantID) AS TotalTenants FROM Tenants GROUP BY Gender;

```
mysql> SELECT Gender, COUNT(TenantID) AS TotalTenants FROM Tenants GROUP BY Gender;
+-----+-----+
| Gender | TotalTenants |
+-----+-----+
| Male | 5 |
| Female | 5 |
+-----+-----+
2 rows in set (0.00 sec)
```

2. Group by Tenant

SELECT TenantID, SUM(RentAmount) AS TotalRentPaid FROM RentPayments GROUP BY TenantID;

```
mysql> SELECT TenantID, SUM(AmountPaid) AS TotalRentPaid FROM RentPayments GROUP BY TenantID;
+-----+-----+
| TenantID | TotalRentPaid |
+-----+-----+
| 1 | 12000.00 |
| 2 | 15000.00 |
| 3 | 17500.00 |
| 4 | 14200.00 |
| 5 | 20500.00 |
| 6 | 24750.00 |
| 7 | 16800.00 |
| 8 | 15800.00 |
| 9 | 16250.00 |
| 10 | 22500.00 |
+-----+-----+
10 rows in set (0.01 sec)
```

3. Group by Status

SELECT Status, COUNT(RequestID) AS TotalRequests FROM MaintenanceRequests GROUP BY Status;

```
mysql> SELECT Status, COUNT(RequestID) AS TotalRequests FROM MaintenanceRequests GROUP BY Status;
+-----+-----+
| Status | TotalRequests |
+-----+-----+
| Resolved | 4 |
| In Progress | 4 |
| Pending | 2 |
+-----+-----+
3 rows in set (0.01 sec)
```

ORDER BY:

1. Tenants Ordered Alphabetically by Name

Problem: Display all tenants **sorted alphabetically by their names**.

SELECT TenantName FROM Tenants ORDER BY TenantName ASC;

```
mysql>
mysql> SELECT TenantName FROM Tenants ORDER BY TenantName ASC;
+-----+
| TenantName |
+-----+
| Aarohi Joshi |
| Amit Verma |
| Arjun Das |
| Karan Patel |
| Nisha Rao |
| Priya Nair |
| Riya Shah |
| Rohit Gupta |
| Sneha Mehta |
| Vikas Sharma |
+-----+
10 rows in set (0.00 sec)
```

2. Top 5 Tenants Paying Highest AmountPaid

Problem: Display **top 5 tenants** paying the **highest amount paid**.

SELECT TenantID FROM RentPayments ORDER BY AmountPaid DESC LIMIT 5;

```
mysql> SELECT TenantID FROM RentPayments ORDER BY AmountPaid DESC LIMIT 5;
+-----+
| TenantID |
+-----+
|         6 |
|        10 |
|         5 |
|         3 |
|         7 |
+-----+
5 rows in set (0.00 sec)
```

AGGREGATE FUNCTIONS:

1. Find Total Number of Properties

Problem: Count how many properties are available in the system.

SELECT COUNT(PropertyID) AS Total_Properties FROM Properties;

```
mysql>
mysql> SELECT COUNT(PropertyID) AS Total_Properties FROM Properties;
+-----+
| Total_Properties |
+-----+
|                10 |
+-----+
1 row in set (0.00 sec)
```

2. Calculate Total Rent Collected

Problem: Find the total rent collected from all tenants.

SELECT SUM(AmountPaid) AS Total_Rent_Collected FROM RentPayments;

```
mysql> SELECT SUM(AmountPaid) AS Total_Rent_Collected FROM RentPayments;
+-----+
| Total_Rent_Collected |
+-----+
|          175300.00 |
+-----+
1 row in set (0.00 sec)
```

3. Find Average Monthly Rent by Location

Problem: Show the average rent for each city/location.

SELECT Location, AVG(MonthlyRent) AS Average_Rent FROM Properties GROUP BY Location;

```
mysql> SELECT Location, AVG(MonthlyRent) AS Average_Rent FROM Properties GROUP BY Location;
+-----+-----+
| Location | Average_Rent |
+-----+-----+
| Mumbai   | 14000.000000 |
| Pune     | 15000.000000 |
| Goa      | 18000.000000 |
| Lonavala | 14000.000000 |
| Delhi    | 20000.000000 |
| Bangalore | 25000.000000 |
| Hyderabad | 17000.000000 |
| Chennai  | 15500.000000 |
| Jaipur   | 16000.000000 |
| Ahmedabad | 22000.000000 |
+-----+-----+
10 rows in set (0.00 sec)
```

4. Find Maximum and Minimum Rent Paid

Problem: Display the highest and lowest rent amounts paid.

SELECT MAX(AmountPaid) AS Highest_Rent, MIN(AmountPaid) AS Lowest_Rent FROM RentPayments;

```
mysql> SELECT MAX(AmountPaid) AS Highest_Rent, MIN(AmountPaid) AS Lowest_Rent FROM RentPayments;
+-----+-----+
| Highest_Rent | Lowest_Rent |
+-----+-----+
| 24750.00 | 12000.00 |
+-----+-----+
1 row in set (0.01 sec)
```

5. Count Maintenance Requests by Status

Problem: Count how many maintenance requests are *Pending*, *In Progress*, and *Resolved*.

SELECT Status, COUNT(RequestID) AS Total_Requests FROM MaintenanceRequests GROUP BY Status;

```
mysql> SELECT Status, COUNT(RequestID) AS Total_Requests FROM MaintenanceRequests
-> GROUP BY Status;
+-----+-----+
| Status | Total_Requests |
+-----+-----+
| Resolved | 4 |
| In Progress | 4 |
| Pending | 2 |
+-----+-----+
3 rows in set (0.00 sec)
```

SUBQUERY:

1. Find Tenants Paying Rent Above the Average

Problem: Display tenants who pay more than the average rent.

```
SELECT TenantID, TenantName
FROM Tenants
WHERE TenantID IN (
    SELECT TenantID
    FROM RentPayments
    WHERE AmountPaid > (SELECT AVG(AmountPaid) FROM RentPayments)
);
```

```
mysql> SELECT TenantID, TenantName
-> FROM Tenants
-> WHERE TenantID IN (
->     SELECT TenantID
->     FROM RentPayments
->     WHERE AmountPaid > (SELECT AVG(AmountPaid) FROM RentPayments)
-> );
```

TenantID	TenantName
5	Arjun Das
6	Priya Nair
10	Aarohi Joshi

3 rows in set (0.01 sec)

2. Find Properties Owned by Owners from a Specific City

Problem: List property names owned by owners who have properties located in *Pune*.

```
SELECT PropertyName
FROM Properties
WHERE OwnerID IN (
    SELECT OwnerID
    FROM Properties
    WHERE Location = 'Pune'
);
```

```
mysql> SELECT PropertyName
-> FROM Properties
-> WHERE OwnerID IN (
->     SELECT OwnerID
->     FROM Properties
->     WHERE Location = 'Pune'
-> );
+-----+
| PropertyName |
+-----+
| Green Valley Homes |
+-----+
1 row in set (0.01 sec)
```

3. Find Tenants Who Haven't Made Any Rent Payment

Problem: Display tenants who have not yet paid rent.

```
SELECT TenantName
FROM Tenants
WHERE TenantID NOT IN (
    SELECT TenantID
    FROM RentPayments
);
```

```
mysql> SELECT TenantName
-> FROM Tenants
-> WHERE TenantID NOT IN (
->     SELECT TenantID
->     FROM RentPayments
-> );
Empty set (0.00 sec)
```

4. Find Owner with the Highest Property Rent

Problem: Display the owner who owns the property with the highest rent.

```
SELECT OwnerName
FROM Owners
WHERE OwnerID = (
    SELECT OwnerID
    FROM Properties
    WHERE MonthlyRent = (SELECT MAX(MonthlyRent) FROM Properties)
);
```

```
mysql> SELECT OwnerName
-> FROM Owners
-> WHERE OwnerID = (
->     SELECT OwnerID
->     FROM Properties
->     WHERE MonthlyRent = (SELECT MAX(MonthlyRent) FROM Properties)
-> );
+-----+
| OwnerName |
+-----+
| Ketki Joshi |
+-----+
1 row in set (0.00 sec)
```

5. Find Maintenance Requests of the Tenant Who Paid the Maximum Rent

Problem: Show all maintenance requests made by the tenant who paid the highest rent.

```
SELECT * FROM MaintenanceRequests
WHERE TenantID = (
    SELECT TenantID
    FROM RentPayments
    WHERE AmountPaid = (SELECT MAX(AmountPaid) FROM RentPayments));
```

```
mysql> SELECT *
-> FROM MaintenanceRequests
-> WHERE TenantID = (
->     SELECT TenantID
->     FROM RentPayments
->     WHERE AmountPaid = (SELECT MAX(AmountPaid) FROM RentPayments)
-> );
+-----+-----+-----+-----+-----+
| RequestID | TenantID | RequestDate | IssueDescription | Status |
+-----+-----+-----+-----+-----+
| 6 | 6 | 2025-10-05 | Door lock issue | Resolved |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

JOINS:

1. List All Tenants with Their Property Details

```
SELECT t.TenantName, t.Email, p.PropertyName, p.Location, p.MonthlyRent
FROM Tenants t
INNER JOIN Properties p ON t.PropertyID = p.PropertyID;
```

```
mysql>
mysql> SELECT t.TenantName, t.Email, p.PropertyName, p.Location, p.MonthlyRent
-> FROM Tenants t
-> INNER JOIN Properties p ON t.PropertyID = p.PropertyID;
```

TenantName	Email	PropertyName	Location	MonthlyRent
Amit Verma	amitv@gmail.com	Sunrise Apartments	Mumbai	14000.00
Riya Shah	riyashah@gmail.com	Green Valley Homes	Pune	15000.00
Karan Patel	karanp@gmail.com	Ocean View Residency	Goa	18000.00
Sneha Mehta	snehamehta@gmail.com	Hilltop Towers	Lonavala	14000.00
Arjun Das	arjundas@gmail.com	Maple Residency	Delhi	20000.00
Priya Nair	priyanair@gmail.com	Royal Gardens	Bangalore	25000.00
Rohit Gupta	rohitgupta@gmail.com	Riverstone Villas	Hyderabad	17000.00
Nisha Rao	nisharao@gmail.com	Golden Nest Apartments	Chennai	15500.00
Vikas Sharma	vikassharma@gmail.com	Palm Paradise	Jaipur	16000.00
Aarohi Joshi	aarohijoshi@gmail.com	Silverline Heights	Ahmedabad	22000.00

```
10 rows in set (0.00 sec)
```

2. Display All Properties and Their Owners (Even If No Tenant Yet)

```
SELECT p.PropertyName, p.Location, o.OwnerName, o.Email
FROM Properties p
LEFT JOIN Owners o ON p.OwnerID = o.OwnerID;
```

```
mysql> SELECT p.PropertyName, p.Location, o.OwnerName, o.Email
-> FROM Properties p
-> LEFT JOIN Owners o ON p.OwnerID = o.OwnerID;
```

PropertyName	Location	OwnerName	Email
Sunrise Apartments	Mumbai	Ramesh Patel	ramesh@gmail.com
Green Valley Homes	Pune	Meenakshi Patil	meenakshi@gmail.com
Ocean View Residency	Goa	Jay Aggrawal	jay@gmail.com
Hilltop Towers	Lonavala	Ashish Bhatiya	ashish@gmail.com
Maple Residency	Delhi	Rahul Singh	rahul@gmail.com
Royal Gardens	Bangalore	Ketki Joshi	ketki@gmail.com
Riverstone Villas	Hyderabad	Sanvi Mathur	sanvi@gmail.com
Golden Nest Apartments	Chennai	Laxmi Bajaj	laxmi@gmail.com
Palm Paradise	Jaipur	Arvind Rao	arvind@gmail.com
Silverline Heights	Ahmedabad	Deepika Singh	deepika@gmail.com

```
10 rows in set (0.00 sec)
```

3. Show Rent Payments with Tenant Names

```
SELECT r.PaymentID, t.TenantName, r.PaymentDate, r.AmountPaid, r.PaymentMode
FROM RentPayments r
INNER JOIN Tenants t ON r.TenantID = t.TenantID;
```

```
mysql>
mysql> SELECT r.PaymentID, t.TenantName, r.PaymentDate, r.AmountPaid, r.PaymentMode
-> FROM RentPayments r
-> INNER JOIN Tenants t ON r.TenantID = t.TenantID;
```

PaymentID	TenantName	PaymentDate	AmountPaid	PaymentMode
1	Amit Verma	2025-10-01	12000.00	Online
2	Riya Shah	2025-10-03	15000.00	Online
3	Karan Patel	2025-09-28	17500.00	Online
4	Sneha Mehta	2025-10-02	14200.00	Online
5	Arjun Das	2025-10-04	20500.00	Online
6	Priya Nair	2025-09-30	24750.00	Online
7	Rohit Gupta	2025-10-05	16800.00	Online
8	Nisha Rao	2025-10-06	15800.00	Online
9	Vikas Sharma	2025-10-07	16250.00	Online
10	Aarohi Joshi	2025-10-08	22500.00	Online

```
10 rows in set (0.00 sec)
```

4. Display All Tenants and Their Maintenance Requests (Include Tenants Without Requests)

```
SELECT t.TenantName, m.RequestID, m.IssueDescription, m.Status
FROM Tenants t
LEFT JOIN MaintenanceRequests m ON t.TenantID = m.TenantID;
```

```
mysql>
mysql> SELECT t.TenantName, m.RequestID, m.IssueDescription, m.Status
-> FROM Tenants t
-> LEFT JOIN MaintenanceRequests m ON t.TenantID = m.TenantID;
```

TenantName	RequestID	IssueDescription	Status
Amit Verma	1	Leaking tap in kitchen	Resolved
Riya Shah	2	Air conditioner not cooling	In Progress
Karan Patel	3	Broken window in living room	Pending
Sneha Mehta	4	Power outage in bedroom	Resolved
Arjun Das	5	Water leakage in bathroom	Pending
Priya Nair	6	Door lock issue	Resolved
Rohit Gupta	7	Internet connection unstable	In Progress
Nisha Rao	8	Wall paint peeling off	In Progress
Vikas Sharma	9	Ceiling fan making noise	Resolved
Aarohi Joshi	10	Kitchen sink clogged	In Progress

```
10 rows in set (0.00 sec)
```

5. Show Owners Whose Properties Have Been Rented by Tenants

```
SELECT o.OwnerName, p.PropertyName, t.TenantName
FROM Owners o
INNER JOIN Properties p ON o.OwnerID = p.OwnerID
INNER JOIN Tenants t ON p.PropertyID = t.PropertyID;
```

```
mysql>
mysql> SELECT o.OwnerName, p.PropertyName, t.TenantName
-> FROM Owners o
-> INNER JOIN Properties p ON o.OwnerID = p.OwnerID
-> INNER JOIN Tenants t ON p.PropertyID = t.PropertyID;
+-----+-----+-----+
| OwnerName | PropertyName | TenantName |
+-----+-----+-----+
| Ramesh Patel | Sunrise Apartments | Amit Verma |
| Meenakshi Patil | Green Valley Homes | Riya Shah |
| Jay Aggrawal | Ocean View Residency | Karan Patel |
| Ashish Bhatiya | Hilltop Towers | Sneha Mehta |
| Rahul Singh | Maple Residency | Arjun Das |
| Ketki Joshi | Royal Gardens | Priya Nair |
| Sanvi Mathur | Riverstone Villas | Rohit Gupta |
| Laxmi Bajaj | Golden Nest Apartments | Nisha Rao |
| Arvind Rao | Palm Paradise | Vikas Sharma |
| Deepika Singh | Silverline Heights | Aarohi Joshi |
+-----+-----+-----+
10 rows in set (0.00 sec)
```

String-Based SQL Queries:

1. Find Tenants Whose Names Start with 'R'

Problem: Display tenant names that begin with the letter **R**.

```
SELECT TenantName, Email FROM Tenants WHERE TenantName LIKE 'R%';
```

```
mysql>
mysql> SELECT TenantName, Email FROM Tenants WHERE TenantName LIKE 'R%';
+-----+-----+
| TenantName | Email |
+-----+-----+
| Riya Shah | riyashah@gmail.com |
| Rohit Gupta | rohitgupta@gmail.com |
+-----+-----+
2 rows in set (0.00 sec)
```

2. Show Owners Whose Email Ends with 'gmail.com'

Problem: List owners who have Gmail accounts.

```
SELECT OwnerName, Email FROM Owners WHERE Email LIKE '%@gmail.com';
```

```
mysql> SELECT OwnerName, Email FROM Owners WHERE Email LIKE '%@gmail.com';
+-----+-----+
| OwnerName | Email |
+-----+-----+
| Ramesh Patel | ramesh@gmail.com |
| Meenakshi Patil | meenakshi@gmail.com |
| Jay Aggrawal | jay@gmail.com |
| Ashish Bhatiya | ashish@gmail.com |
| Rahul Singh | rahul@gmail.com |
| Ketki Joshi | ketki@gmail.com |
| Sanvi Mathur | sanvi@gmail.com |
| Laxmi Bajaj | laxmi@gmail.com |
| Arvind Rao | arvind@gmail.com |
| Deepika Singh | deepika@gmail.com |
+-----+-----+
10 rows in set (0.00 sec)
```

3. Tenants with Short Names

Problem: List tenants whose names are **less than 6 characters long**.

```
SELECT TenantName FROM Tenants WHERE LENGTH(TenantName) < 6;
```

```
mysql> SELECT TenantName FROM Tenants WHERE LENGTH(TenantName) < 6;
Empty set (0.00 sec)
```

4. Combine Tenant Name and Contact into One Column

Problem: Display tenant details with full contact info.

```
SELECT CONCAT(TenantName, ' - ', PhoneNumber) AS TenantContact FROM Tenants;
```

```
mysql>
mysql> SELECT CONCAT(TenantName, ' - ', PhoneNumber) AS TenantContact FROM Tenants;
+-----+
| TenantContact |
+-----+
| Amit Verma - 9811112233 |
| Riya Shah - 9822334455 |
| Karan Patel - 9833445566 |
| Sneha Mehta - 9844556677 |
| Arjun Das - 9855667788 |
| Priya Nair - 9866778899 |
| Rohit Gupta - 9877889900 |
| Nisha Rao - 9888990011 |
| Vikas Sharma - 9899001122 |
| Aarohi Joshi - 9900112233 |
+-----+
10 rows in set (0.00 sec)
```

5. Convert Property Names to Uppercase

Problem: Show all property names in uppercase format.

```
SELECT UPPER(PropertyName) AS Property_Name_Upper FROM Properties;
```

```
mysql> SELECT UPPER(PropertyName) AS Property_Name_Upper FROM Properties;
+-----+
| Property_Name_Upper |
+-----+
| SUNRISE APARTMENTS  |
| GREEN VALLEY HOMES  |
| OCEAN VIEW RESIDENCY |
| HILLTOP TOWERS      |
| MAPLE RESIDENCY     |
| ROYAL GARDENS       |
| RIVERSTONE VILLAS   |
| GOLDEN NEST APARTMENTS |
| PALM PARADISE       |
| SILVERLINE HEIGHTS  |
+-----+
10 rows in set (0.01 sec)
```

5. Implement a Trigger, a Function, and a Stored Procedure relevant to your system's logic.

1. Trigger

Problem: Automatically update the tenant's total rent paid whenever a new rent payment is inserted into the RentPayments table.

```
alter table Tenants add TotalRentPaid decimal(10,2) default 0;
```

```
mysql> select TenantID, TenantName, TotalRentPaid from Tenants;
+-----+-----+-----+
| TenantID | TenantName | TotalRentPaid |
+-----+-----+-----+
| 1 | Amit Verma | 0.00 |
| 2 | Riya Shah | 0.00 |
| 3 | Karan Patel | 0.00 |
| 4 | Sneha Mehta | 0.00 |
| 5 | Arjun Das | 0.00 |
| 6 | Priya Nair | 0.00 |
| 7 | Rohit Gupta | 0.00 |
| 8 | Nisha Rao | 0.00 |
| 9 | Vikas Sharma | 0.00 |
| 10 | Aarohi Joshi | 0.00 |
+-----+-----+-----+
10 rows in set (0.00 sec)
```

create trigger after_rent_payment

after insert on RentPayments

for each row

update Tenants

set TotalRentPaid = TotalRentPaid + new.AmountPaid

where TenantID = new.TenantID;

```
mysql> show triggers;
+-----+-----+-----+-----+
| Trigger          | Event | Table      | Statement |
+-----+-----+-----+-----+
| after_rent_payment | INSERT | rentpayments | update Tenants
set TotalRentPaid = TotalRentPaid + new.AmountPaid
where TenantID = new.TenantID | AFTER | 2025-10-15 08:08:59.17 |
neral_ci          | utf8mb4_0900_ai_ci |
+-----+-----+-----+-----+
1 row in set (0.01 sec)
```

insert into RentPayments (PaymentID, TenantID, PaymentDate, AmountPaid, PaymentMode)

values (201, 1, '2025-10-15', 12000.00, 'Online');

```
mysql> select * from rentpayments;
+-----+-----+-----+-----+-----+
| PaymentID | TenantID | PaymentDate | AmountPaid | PaymentMode |
+-----+-----+-----+-----+-----+
| 1         | 1        | 2025-10-01  | 12000.00   | Online      |
| 2         | 2        | 2025-10-03  | 15000.00   | Cash        |
| 3         | 3        | 2025-09-28  | 17500.00   | Cheque      |
| 4         | 4        | 2025-10-02  | 14200.00   | Online      |
| 5         | 5        | 2025-10-04  | 20500.00   | Cash        |
| 6         | 6        | 2025-09-30  | 24750.00   | Online      |
| 7         | 7        | 2025-10-05  | 16800.00   | Cash        |
| 8         | 8        | 2025-10-06  | 15800.00   | Cheque      |
| 9         | 9        | 2025-10-07  | 16250.00   | Online      |
| 10        | 10       | 2025-10-08  | 22500.00   | Cash        |
| 201       | 1        | 2025-10-15  | 12000.00   | Online      |
+-----+-----+-----+-----+-----+
11 rows in set (0.00 sec)
```

2. Function

Problem: Create a function that returns a rent grade based on the amount paid by the tenant.

delimiter \$\$

create function get_rent_grade(amount decimal(10,2))

returns varchar(10)

deterministic

begin

declare grade varchar(10);

if amount >= 20000 then

set grade = 'A';

elseif amount >= 15000 then

set grade = 'B';

elseif amount >= 10000 then

set grade = 'C';

else

set grade = 'D';

end if;

return grade;

end \$\$

delimiter ;

```
mysql> select get_rent_grade(22000);
+-----+
| get_rent_grade(22000) |
+-----+
| A                     |
+-----+
1 row in set (0.00 sec)

mysql> select get_rent_grade(10500);
+-----+
| get_rent_grade(10500) |
+-----+
| C                     |
+-----+
1 row in set (0.00 sec)
```

3. Stored Procedure

Problem: Display the total rent collected from all tenants for a specific property owner.

delimiter \$\$

```
create procedure get_total_rent_by_owner(in input_ownerid int, out total_rent
decimal(10,2))
```

```
begin
```

```
    select sum(r.AmountPaid)
```

```
    into total_rent
```

```
    from RentPayments r
```

```
    join Tenants t on r.TenantID = t.TenantID
```

```
    join Properties p on t.PropertyID = p.PropertyID
```

```
    where p.OwnerID = input_ownerid;
```

```
end $$
```

delimiter ;

```
mysql> call get_total_rent_by_owner(3, @total);
Query OK, 1 row affected (0.00 sec)
```

```
mysql> select @total as Total_Rent_Collected;
```

Total_Rent_Collected
17500.00

```
1 row in set (0.00 sec)
```

6. Normalization

Normalize your database up to Third Normal Form (3NF) and provide a short explanation. • Identify repeating groups → convert to 1NF. • Remove partial dependencies → convert to 2NF. • Remove transitive dependencies → convert to 3NF. • Clearly show the final normalized tables. • Explain each step briefly

Final Normalized Tables

Owners (OwnerID, OwnerName, ContactNumber, Email)

Properties (PropertyID, PropertyName, Location, MonthlyRent, OwnerID)

RentPayments (PaymentID, TenantID, PaymentDate, AmountPaid, PaymentMode)

MaintenanceRequests (RequestID, TenantID, RequestDate, IssueDescription, Status)

Tenants (TenantID, TenantName, Gender, PhoneNumber, Email, PropertyID, TotalRentPaid)

Step 1: Achieving First Normal Form (1NF)

Goal: Eliminate repeating groups and ensure all values are atomic

Explanation: The first step is to ensure each column holds a single, indivisible value and that we aren't trying to store a list of items within a single field.

In our design: Instead of having a “tenants” or “payments” column inside the Properties table, which would be a repeating group, we created separate **Tenants** and **RentPayments** tables. Each tenant and each payment event gets its own unique row. This makes the data structured and easily queryable. All columns, like **OwnerName**, **PropertyName**, and **TenantName**, are broken down into their smallest meaningful parts (atomic).

Step 2: Achieving Second Normal Form (2NF)

Goal: Be in 1NF and remove any partial dependencies.

Explanation: A partial dependency occurs when a non-key attribute depends on only a part of a composite primary key.

In our design: Most of our tables use a single-column primary key (e.g., **OwnerID** or **TenantID**), so they inherently satisfy 2NF as there's no "part" of the key for an attribute to depend on. The one table that could naturally have a composite key is **RentPayments**, where (**TenantID**, **PaymentDate**) could uniquely identify a row. The **AmountPaid** and **PaymentMode** attributes depend on both the tenant and the payment date—they cannot be determined by just one of them. Since these attributes are fully dependent on the entire key, there are no partial dependencies.

Step 3: Achieving Third Normal Form (3NF)

Goal: Be in 2NF and remove any transitive dependencies.

Explanation: A transitive dependency is when a non-key attribute depends on another non-key attribute, rather than directly on the primary key. Essentially, we want to make sure that all attributes in a table are about the primary key and nothing else.

In our design: We achieve this by separating entities into their own tables. For example, in the **Tenants** table, we only store the **PropertyID** as a reference. We do not store the property's name or location. If we did, **Location** would depend on **PropertyName**, which in turn depends on **TenantID**. This indirect dependency (**TenantID** → **PropertyName** → **Location**) is a transitive dependency. By creating the **Properties** table, we ensure that **PropertyName** and **Location** depend only on their primary key, **PropertyID**. This makes the data easy to maintain; if a property's location changes, we only have to update it in one place.